

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC MỞ THÀNH PHỐ HỒ CHÍ MINH**



NGUYỄN SỸ HÀ

**PHÁT HIỆN DEEPPFAKE
DỰA TRÊN HỌC MÁY SÂU**

**ĐỒ ÁN NGÀNH
NGÀNH KHOA HỌC MÁY TÍNH**

TP. HỒ CHÍ MINH, 2024

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC MỞ THÀNH PHỐ HỒ CHÍ MINH



NGUYỄN SỸ HÀ

PHÁT HIỆN DEEPPFAKE
DỰA TRÊN HỌC MÁY SÂU

Mã số sinh viên: 2151013018

ĐỒ ÁN NGÀNH
NGÀNH KHOA HỌC MÁY TÍNH

Giảng viên hướng dẫn: Lê Viết Tuấn

TP. HỒ CHÍ MINH, 2024

LỜI CẢM ƠN

Trước tiên, em xin gửi lời cảm ơn sâu sắc đến thầy Lê Viết Tuấn là người đã luôn tận tình chỉ dẫn cho em trong suốt quá trình thực hiện môn đồ án ngành này. Những lời khuyên quý báu và sự hướng dẫn tận tâm của thầy đã giúp em hoàn môn một cách tốt nhất.

Em cũng xin chân thành cảm ơn đến các anh chị và bạn bè đã luôn sẵn lòng hỗ trợ, động viên và chia sẻ kinh nghiệm trong quá trình em thực hiện đề tài. Sự giúp đỡ nhiệt tình đó đã tạo điều kiện thuận lợi để em vượt qua những khó khăn và thử thách.

Đặc biệt, em muốn dành lời cảm ơn chân thành đến gia đình, bạn bè những người đã luôn là điểm tựa vững chắc về tinh thần, ủng hộ và tạo điều kiện tốt nhất cho em trong suốt quá trình học tập và nghiên cứu.

Dù đã cố gắng hết sức, đồ án vẫn không tránh khỏi những thiếu sót. Em rất mong nhận được sự góp ý từ thầy và các bạn để có thể hoàn thiện tốt hơn trong tương lai.

NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

TÓM TẮT ĐỒ ÁN NGÀNH

Chủ đề **nhận diện deepfake** [1] bằng **học máy sâu** [2] ngày càng trở nên quan trọng khi các công nghệ số phát triển vượt bậc. Deepfake [1] là các video giả mạo do học máy tạo ra, gây ra nhiều vấn đề nghiêm trọng về an ninh, lừa đảo, và vi phạm quyền riêng tư. Mục tiêu của việc sử dụng các mô hình học sâu là xây dựng hệ thống có khả năng phát hiện chính xác các dấu hiệu bất thường trong video, giúp phân biệt giữa video thật và giả. Điều này góp phần bảo vệ quyền lợi cá nhân, giảm thiểu rủi ro về bảo mật thông tin, ngăn chặn các hành vi lừa đảo tinh vi và đảm bảo tính xác thực của nội dung số trong thời đại hiện nay.

Phương pháp mà em đề xuất là sử dụng mạng nơ-ron tích chập **ResNeXt101** [3], một phiên bản nâng cấp từ **ResNeXt50** [4], để trích xuất các đặc trưng không gian từ từng khung hình của video. Sau đó, các đặc trưng này sẽ được chuyển vào mạng **LSTM** [5] [6] (**Long Short-Term Memory**), một biến thể của **RNN (Recurrent Neural Network)** [5], để phân tích sự phụ thuộc theo chuỗi thời gian giữa các khung hình. Từ đó, mô hình sẽ xác định xem video có bị làm giả hay không, giúp phân loại giữa video deepfake [1] và video thật.

Để đa dạng hóa tập dữ liệu làm tăng tính ứng dụng trong thực tiễn, em đã thu thập nhiều bộ dữ liệu lớn và phong phú có sẵn như **FaceForensics++** [7], **Deepfake Detection Challenge** [8], và **Celeb-DF** [9]. Kết quả cho thấy mô hình đạt độ chính xác cao trong việc phát hiện deepfake và hiệu quả khi ứng dụng trong các tình huống thời gian thực. Phương pháp này không chỉ đơn giản mà còn mạnh mẽ hơn so với mô hình **ResNeXt50** trong việc phát hiện các video giả mạo.

ABSTRACT

The topic of **deepfake detection** [1] using **deep learning** [2] has become increasingly important as digital technologies continue to evolve rapidly. Deepfakes are fabricated videos generated by machine learning algorithms, raising significant concerns regarding security, fraud, and privacy violations. The goal of using deep learning models is to develop a system capable of accurately detecting video irregularities, helping to distinguish between real and fake footage. This contributes to protecting individual rights, minimizing the risks of data security breaches, preventing sophisticated fraudulent activities, and ensuring the authenticity of digital content in today's world.

The method I propose is to use the **ResNeXt101** [3] convolutional neural network, an upgraded version of **ResNeXt50** [4], to extract spatial features from each video frame. These features will then be passed into the **LSTM (Long Short-Term Memory)** [5] [6] network, a variant of the **RNN (Recurrent Neural Network)** [5], to analyze the temporal dependencies between frames. From there, the model will determine whether the video is faked, helping to classify between deepfake and real videos.

To diversify the dataset and enhance real-world applicability, I collected multiple large and comprehensive datasets such as **FaceForensics++** [7], **Deepfake Detection Challenge** [8], and **Celeb-DF** [9]. The results show that the model achieved high accuracy in detecting deepfakes and proved effective when applied to real-time scenarios. This approach is not only simple but also more powerful than ResNeXt50 in identifying fake videos.

MỤC LỤC

DANH MỤC TỪ VIẾT TẮT	viii
DANH MỤC HÌNH VẼ.....	ix
DANH MỤC BẢNG	x
CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI.....	1
1.1. Giới thiệu	1
1.2. Mục tiêu đề tài	1
1.3. Lý do chọn đề tài	2
1.4. Bố cục báo cáo.....	2
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT	3
2.1. Deep Learning	3
2.1.1. Khái niệm	3
2.1.2. Đặc điểm của Deep Learning	3
2.2. Mạng tích chập nơ-ron r tích chập (CNN).....	5
2.2.1. Khái niệm	5
2.2.2. Đặc điểm.....	5
2.2.3. Hàm kích hoạt ReLu.....	7
2.2.4. Hàm kích hoạt Softmax	8
2.2.5. Hàm mất mát (Loss Function).....	9
2.3. Mạng nơ-ron hồi quy (RNN)	9
2.3.1. Khái niệm	9
2.3.2. Đặc điểm.....	9
2.3.3. Các dạng bài toán của RNN	10
2.3.4. Mạng sinh đối nghịch (GAN).....	10
CHƯƠNG 3. KIẾN TRÚC MẠNG NƠ RON ResNeXt101 VÀ LSTM	12

3.1. Mô hình ResNet (Residual Networks)	12
3.2. Mô hình ResNext CNN	13
3.3. Mô hình ResNext CNN ResNext50	13
3.4. Mô hình ResNext CNN ResNext101	15
3.5. Mạng nơ-ron hồi quy LSTM trong việc xử lý chuỗi.....	16
3.5.1. Khái niệm	16
3.5.2. Cấu trúc của một tầng LSTM	16
3.5.3. Quá trình xử lý trong LSTM	17
3.5.4. Dropout trong mạng LSTM.....	18
3.5.5. Tầng chú ý(Attention Layer) [18]	19
3.6. LSTM 2 chiều (Bidirectional LSTM).....	20
3.6.1. Kiến trúc của BiLSTM	20
3.6.2. Ưu điểm của BiLSTM.....	22
3.6.3. Ứng dụng	22
CHƯƠNG 4. XÂY DỰNG HỆ THỐNG	23
4.1. Tổng quan về hệ thống	23
4.2. Chuẩn bị dữ liệu	24
4.3. Tiền xử lý dữ liệu	25
4.4. Phân chia tập dữ liệu	27
4.5. Xây Dựng kiến trúc mô hình	27
4.5.1. Import các thư viện cần thiết:	27
4.5.2. Định nghĩa lớp AttentionLayer:	28
4.5.3. Định nghĩa lớp HighAccuracyVideoClassifier:	29
4.5.4. Phương thức forward của HighAccuracyVideoClassifier.....	30
4.5.5. Hàm loss tùy chỉnh	31

4.5.6. Khởi tạo mô hình và optimizer.....	31
4.6. Huấn luyện và đánh giá mô hình.....	32
4.6.1. Huấn luyện mô hình	32
4.6.2. Kiểm tra mô hình.....	33
4.6.3. Theo dõi lưu trữ metrics và tính toán độ chính xác.....	34
CHƯƠNG 5. KẾT QUẢ ĐẠT ĐƯỢC.....	36
5.1. Kết quả của mô hình.....	36
5.2. Kết quả khi so sánh với mô hình ResNeXt50	38
CHƯƠNG 6. KẾT LUẬN VÀ PHƯƠNG HƯỚNG PHÁT TRIỂN TRONG	
TƯƠNG LAI.....	39
6.1. Tổng kết các kết quả đạt được.....	39
6.2. Các hạn chế và thách thức còn tồn tại	39
6.3. Phương hướng phát triển trong tương lai	39
6.3.1. Cân bằng hiệu suất và độ chính xác ổn định:.....	39
6.3.2. Kết hợp thêm các mô hình CNN mới hơn:.....	40
6.3.3. Thu thập và bổ sung tập dữ liệu đa dạng hơn:	40

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Giải thích
LSTM	Long short-term memory
CNN	Convolutional Neural Network
RELU	Rectified Linear Unit
RNN	Recurrent Neural Network
GRU	Gated Recurrent Unit
ResNet	Residual Network

DANH MỤC HÌNH VẼ

Hình 2.1 Kiến trúc của CNN	5
Hình 2.2 Ba giai đoạn chính của CNN	6
Hình 2.3 Biểu đồ hàm kích hoạt.....	7
Hình 2.4 Các dạng toán của mô hình RNN.....	10
Hình 2.5 Kiến trúc của GAN	11
Hình 3.1 Mối tương quan giữa độ sâu và hiệu suất mạng.....	12
Hình 3.2 ResNet sử dụng kết nối tắt xuyên qua một hay nhiều lớp.....	13
Hình 3.3 Kiến trúc mô hình ResNext50	14
Hình 3.4 Tải mô hình ResNeXt101_32x8d đã được huấn luyện trước.....	15
Hình 3.5 Kiến trúc tổng quan của LSTM.....	16
Hình 3.6 Tổng quan tầng Dropout.....	19
Hình 3.7 Attention Layer trong mô hình CNN và LSTM	20
Hình 3.8 Kiến trúc của Bidirectional LSTM	21
Hình 4.1 Sơ đồ của hệ thống	23
Hình 4.2 Tập dữ liệu thu thập được.....	25
Hình 4.3 Tiền xử lý dữ liệu	26
Hình 4.4 Phân chia dữ liệu	27
Hình 5.1 Biểu đồ Training và Validation Loss	36
Hình 5.2 Biểu đồ Training và Validation Accuracy.....	37
Hình 5.3 Confusion Matrix: Phân loại Video Fake và Real.....	37

DANH MỤC BẢNG

Bảng 5.1 So sánh kết quả độ chính xác của 2 mô hình	38
---	----

MỞ ĐẦU

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1. Giới thiệu

Deepfake [1] là thuật ngữ dùng để chỉ các video, hình ảnh, hoặc âm thanh được tạo ra bằng trí tuệ nhân tạo (AI) để giả mạo người thật một cách rất thuyết phục. Công nghệ này dựa trên các kỹ thuật **deep learning** (học sâu) như mạng nơ-ron nhân tạo [2] và mô hình Generative Adversarial Networks (GANs) [3], các video Deepfake thường dùng vào mục đích xấu chủ yếu là lan truyền những thông tin sai lệch, mạo danh và tống tiền [1]. Hiện nay các tội phạm công nghệ cao thường sử dụng phương pháp này để thực hiện các hành vi lừa đảo. Chính vì vậy nên em muốn xây dựng một mô hình ở đó người dùng có thể phân biệt được video nào được làm giả hay video thật, điều này có thể giúp mọi người có thể phát hiện được các hành vi lừa đảo hay là những thông tin được lan truyền không đúng sự thật.

1.2. Mục tiêu đề tài

Hiện nay các video deepfake được xây dựng bằng các công cụ như **FaceApp** và **Face Swap**, các công cụ này sử dụng các mạng nơ-ron đã được huấn luyện trước như **GAN** hoặc **Autoencoder** để tạo ra các video giả mạo, vì vậy em áp dụng phương pháp **mạng nơ-ron hồi quy LSTM** để phân tích dữ liệu tuần tự của các khung hình video theo thời gian, kết hợp với **CNN ResNeXt** đã được huấn luyện trước để trích xuất đặc trưng từ từng khung hình trong video [2].

Mạng nơ-ron tích chập **ResNeXt** đảm nhận nhiệm vụ trích xuất các đặc trưng quan trọng từ khung hình, và những đặc trưng này tiếp tục được dùng để huấn luyện **mạng nơ-ron hồi quy dựa trên LSTM** nhằm phân loại video là **deepfake** hay video thật. Để tăng khả năng ứng dụng trong các tình huống thực tế và cải thiện hiệu suất của mô hình trên dữ liệu thời gian thực, em đã huấn luyện phương pháp này với một lượng lớn dữ liệu đa dạng và cân bằng, kết hợp từ nhiều bộ dữ liệu phổ biến như **FaceForensics++** [7], **Deepfake Detection Challenge**, và **Celeb-DF**.

1.3. Lý do chọn đề tài

Hiện nay việc các đối tượng có hành vi thường sẽ gọi điện thoại giả dạng làm công an hay một số cơ quan nhà nước khác bằng các gọi video và sử dụng deepfake thời gian thực nhằm mang tính chất đe dọa những người dân thiếu hiểu biết để lừa đảo, trong đó có cả gia đình và bạn bè của em, nên em mong muốn làm điều gì đó để có thể giảm thiểu các tình trạng hiện nay.

Cơ hội phát triển có thể áp dụng cho các vấn đề như an ninh mạng, pháp luật, truyền thông đến công nghệ, giáo dục và giải trí.

1.4. Bố cục báo cáo

Bài báo cáo sẽ được em sẽ làm bao gồm 6 chương, mỗi chương sẽ có nhiệm vụ khác nhau cụ thể:

Chương 1: Giới thiệu về đề tài: Chương này nêu ra lý do và mục đích chọn đề tài.

Chương 2: Cơ sở lý thuyết: Chương này đề cập đến các khái niệm, chỉ số, và cách hoạt động và lý thuyết liên quan đến đề tài.

Chương 3: Kiến trúc mạng nơ ron ResNeXt101 và LSTM: Chương này đề cập tới khái niệm, cấu trúc và các kiến thức liên quan kiến trúc mạng nơ ron ResNeXt101 và LSTM để có thể sử dụng nhận diện deepfake.

Chương 4: Xây dựng hệ thống: Chương này đề cập về cấu trúc và cách hoạt động của hệ thống sau đó thể.

Chương 5: Kết quả đạt được: Chương này nêu ra các kết quả mà em đã đạt được khi triển khai hệ thống.

Chương 6: Kết luận và phương hướng phát triển trong tương lai: Chương này sẽ tổng kết lại các kết quả đạt được và đề xuất các phương hướng mới để phát triển hệ thống trong tương lai.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. Deep Learning

2.1.1. Khái niệm

Học sâu (Deep Learning) [6] [11] là một nhánh chuyên sâu của **học máy (Machine Learning)**, nơi mà các hệ thống máy tính có khả năng tự học và nâng cao hiệu suất thông qua các thuật toán phức tạp. Khác với các phương pháp truyền thống, học sâu dựa trên các mạng nơ-ron nhân tạo, mô phỏng quá trình xử lý thông tin và tư duy của bộ não con người. Bằng cách sử dụng nhiều lớp nơ-ron, công nghệ này cho phép các mô hình học sâu phân tích và hiểu các đặc trưng ẩn trong dữ liệu một cách hiệu quả hơn, từ đó cải thiện khả năng đưa ra dự đoán và quyết định.

2.1.2. Đặc điểm của Deep Learning

Học sâu (Deep Learning) đã trở thành một bước đột phá quan trọng trong lĩnh vực trí tuệ nhân tạo, cho phép các hệ thống máy tính tự học và nâng cao hiệu suất dựa trên các thuật toán nơ-ron phức tạp. Tuy nhiên, như bất kỳ công nghệ nào, Deep Learning có cả **ưu điểm** và **nhược điểm**, mang lại cả cơ hội và thách thức khi áp dụng trong các lĩnh vực khác nhau.

Ưu điểm của Deep Learning [11]:

- **Kiến trúc mạng nơ-ron linh hoạt** : Một trong những ưu điểm lớn nhất của học sâu là khả năng điều chỉnh kiến trúc mạng nơ-ron theo nhu cầu của từng bài toán [11].
- **Giải quyết các bài toán phức tạp với độ chính xác cao**: Nhờ vào khả năng học các đặc trưng ẩn trong dữ liệu, Deep Learning đã chứng minh hiệu quả vượt trội trong việc giải quyết các bài toán có tính phức tạp cao, chẳng hạn như nhận dạng hình ảnh, dịch ngôn ngữ, và dự đoán chuỗi thời gian. Các mô hình này thường vượt qua các phương pháp học máy truyền thống về độ chính xác [11].

- **Tính tự động hóa cao:** Deep Learning có khả năng tự điều chỉnh, tối ưu hóa mà không cần quá nhiều sự can thiệp từ con người. Các mô hình có thể tự học từ dữ liệu thô, giảm bớt gánh nặng trong việc xử lý dữ liệu và xây dựng đặc trưng thủ công [11].
- **Khả năng tính toán song song và xử lý dữ liệu lớn:** Deep Learning tận dụng khả năng tính toán song song từ các GPU để xử lý lượng dữ liệu lớn trong thời gian ngắn. Điều này làm cho nó đặc biệt phù hợp với các bài toán yêu cầu xử lý dữ liệu lớn như phân tích video, hình ảnh, và các bộ dữ liệu lớn trong thời gian thực [11].

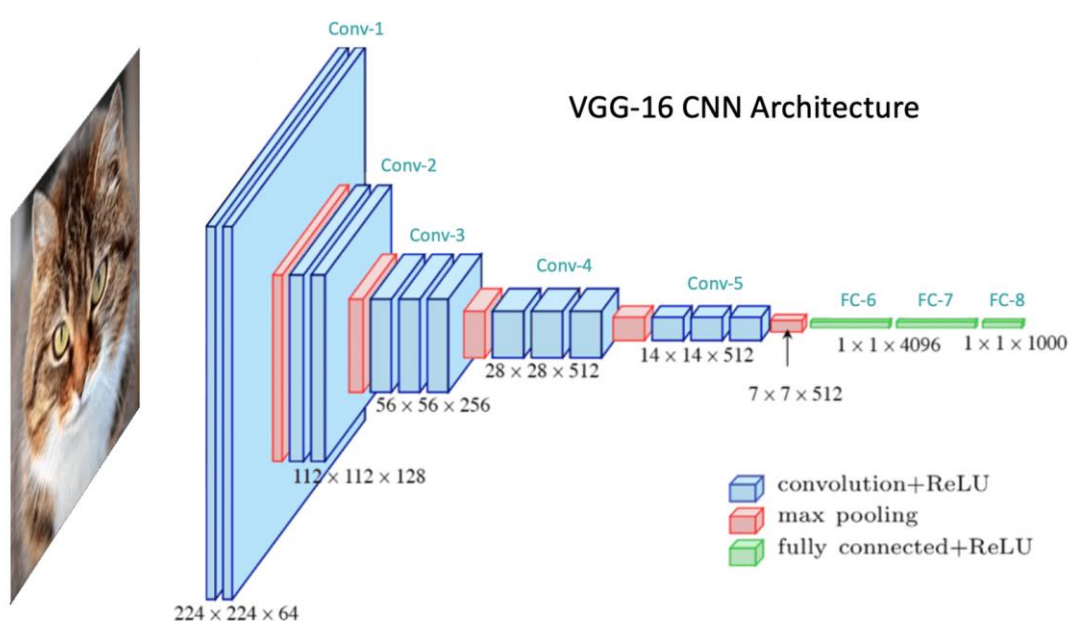
Nhược điểm của Deep Learning [11]:

- **Yêu cầu lượng dữ liệu lớn:** Một trong những khó khăn lớn nhất của Deep Learning là cần một lượng dữ liệu rất lớn để huấn luyện mô hình một cách hiệu quả. Để mô hình hoạt động tối ưu, dữ liệu không chỉ phải lớn mà còn phải có chất lượng cao, điều này gây ra khó khăn trong việc thu thập và chuẩn bị dữ liệu đặc biệt là các đề tài mới lạ để triển khai [11].
- **Chi phí tính toán cao:** Việc huấn luyện các mô hình học sâu phức tạp đòi hỏi phần cứng mạnh mẽ, đặc biệt là GPU, khiến chi phí tính toán trở nên rất cao. Đặc biệt, với các mô hình lớn và dữ liệu khổng lồ, việc huấn luyện và triển khai có thể tiêu tốn nhiều tài nguyên [11].
- **Chưa có nền tảng lý thuyết vững chắc**
Mặc dù Deep Learning đã đạt được nhiều thành tựu, vẫn chưa có một cơ sở lý thuyết toàn diện để hướng dẫn việc tối ưu hóa và lựa chọn các tham số cho mô hình. Điều này có nghĩa là nhiều mô hình phải dựa vào quá trình thử nghiệm và sai sót, tốn nhiều thời gian và tài nguyên [11].
- **Dễ bị ảnh hưởng bởi dữ liệu thiên lệch**
Nếu dữ liệu huấn luyện không cân bằng hoặc có thiên lệch, các mô hình học sâu có thể học theo các thiên lệch đó, dẫn đến kết quả không công bằng hoặc không chính xác. Vấn đề này đặc biệt nguy hiểm trong các ứng dụng liên quan đến xã hội hoặc pháp luật [11].

2.2. Mạng tích chập nơ-ron r tích chập (CNN)

2.2.1. Khái niệm

Mạng nơ-ron tích chập (Convolutional Neural Network – CNN) là một kiến trúc mạng nơ-ron nhân tạo tiên tiến, được thiết kế đặc biệt để giải quyết các bài toán liên quan đến xử lý dữ liệu có cấu trúc lưới, chẳng hạn như hình ảnh và video. CNN hoạt động bằng cách áp dụng các phép **tích chập** (convolution) để trích xuất những đặc trưng quan trọng từ dữ liệu đầu vào, giảm thiểu các yếu tố không cần thiết và làm nổi bật các đặc trưng có ý nghĩa nhất [12].



Hình 2.1 Kiến trúc của CNN

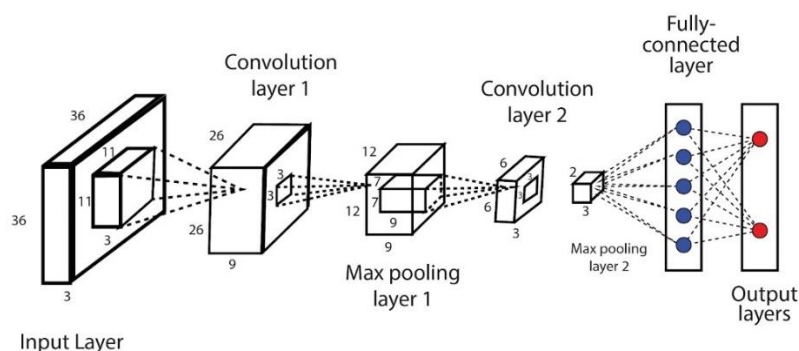
Tích chập là một kỹ thuật trong xử lý tín hiệu số, sử dụng các bộ lọc (filter) hay còn gọi là **kernel** để biến đổi dữ liệu đầu vào, giúp mạng học được các mẫu (pattern) quan trọng như cạnh, góc, hoặc đối tượng trong hình ảnh. Mục tiêu chính là giảm thiểu nhiễu và tập trung vào các đặc điểm quan trọng, từ đó giúp mô hình đưa ra dự đoán chính xác hơn [11].

2.2.2. Đặc điểm

Ngoài các lớp đầu vào (input layer) và đầu ra (output layer), mô hình CNN còn có thêm các lớp lấy mẫu (sampling layer) nhằm giảm số lượng nơ-

ron tham gia vào mỗi lớp tiếp theo. Quá trình xây dựng mô hình CNN thường trải qua ba giai đoạn chính:

- **Giai đoạn tích chập (convolution)** [12]: Thực hiện phép tích chập giữa ma trận đầu vào và các bộ lọc (filters) để tạo thành các tầng đặc trưng mới. Quá trình này thường diễn ra liên tục ở các lớp đầu của mạng và kết hợp với hàm kích hoạt **ReLU** để tăng tính phi tuyến. Mục tiêu là trích xuất các đặc trưng không gian hai chiều từ dữ liệu hình ảnh.
- **Giai đoạn tổng hợp (max pooling)** [12]: Tầng pooling giúp giảm kích thước ma trận bằng cách chọn ra giá trị đại diện từ mỗi vùng mà bộ lọc quét qua. Quá trình này giữ lại các đặc điểm quan trọng của bức ảnh trong khi giảm kích thước dữ liệu, giúp tăng hiệu quả tính toán mà vẫn giữ nguyên các đường nét chính.
- **Giai đoạn kết nối hoàn toàn (fully connected)** [12]: Sau khi dữ liệu đã được giảm kích thước đáng kể, ma trận sẽ được phẳng hóa (flatten) thành một vector, sau đó kết nối với các tầng nơ-ron hoàn toàn (fully connected layers). Lớp kết nối hoàn toàn cuối cùng sẽ chứa số lượng đơn vị bằng với số lớp đầu ra của bài toán.



Hình 2.2 Ba giai đoạn chính của CNN

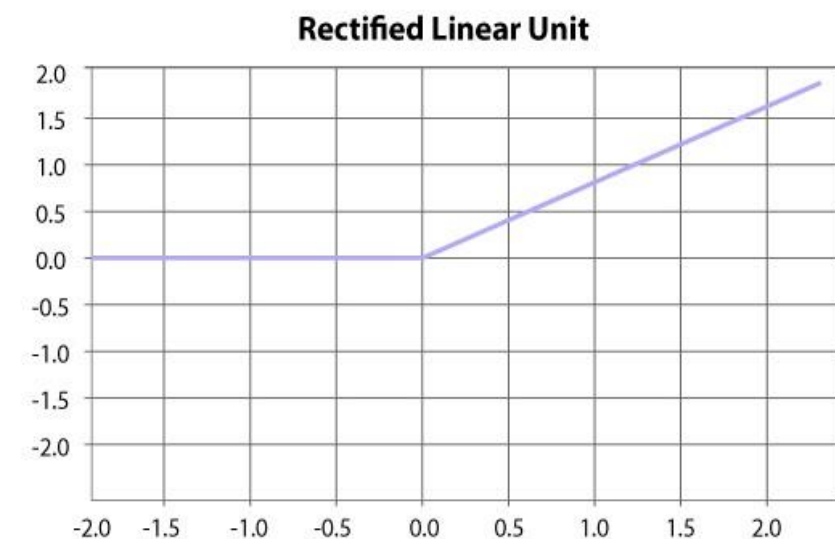
Nhờ những đặc điểm này, CNN thường được ứng dụng rộng rãi trong các lĩnh vực như nhận diện và phân tích hình ảnh, phân tích video, phân đoạn ảnh, và xử lý ngôn ngữ tự nhiên

2.2.3. Hàm kích hoạt ReLU

ReLU (Rectified Linear Unit) là một **hàm kích hoạt** thường được sử dụng trong các mạng nơ-ron nhân tạo, đặc biệt là trong các mô hình mạng nơ-ron sâu (Deep Learning) và mạng tích chập (CNN). ReLU giúp mô hình học các đặc trưng phi tuyến và được định nghĩa đơn giản như sau [13]:

$$f(x) = \max(0, x)$$

Nghĩa là, đầu ra của hàm ReLU bằng **0** nếu đầu vào là số âm, và bằng **chính giá trị đầu vào** nếu đó là số dương.



Hình 2.3 Biểu đồ hàm kích hoạt

Đặc điểm của ReLU:

- **Tính đơn giản:** ReLU có tính toán đơn giản và không phức tạp, giúp tăng tốc quá trình huấn luyện mạng nơ-ron so với các hàm kích hoạt khác như sigmoid hay tanh [13].

- **Giải quyết vấn đề gradient mất dần (vanishing gradient):** Nhờ cấu trúc đơn giản, ReLU không gặp phải vấn đề gradient mất dần, giúp mô hình học tốt hơn, đặc biệt với các mạng có nhiều lớp sâu [12] [13].
- **Tính phi tuyến:** Dù ReLU có một phần tuyến tính với các giá trị dương, nhưng nhờ ngắt quãng tại giá trị $x = 0$, nó vẫn được coi là một hàm phi tuyến, hỗ trợ mô hình học được các quan hệ phức tạp trong dữ liệu [13].

2.2.4. Hàm kích hoạt Softmax

Softmax là một **hàm kích hoạt** thường được sử dụng ở lớp đầu ra của các mô hình mạng nơ-ron, đặc biệt là trong các bài toán phân loại nhiều lớp (multi-class classification). Softmax biến đổi một vector giá trị thực thành một **phân phối xác suất** trên các lớp, với tổng các xác suất bằng 1 [13].

Công thức của Softmax cho một đầu vào z_i trong một tập hợp các giá trị $z = [z_1, z_2, \dots, z_n]$ được định nghĩa là:

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}}$$

Đặc điểm của Softmax:

- **Phân phối xác suất:** Hàm Softmax biến đổi các giá trị thực thành xác suất (giá trị từ 0 đến 1), và tổng của tất cả các xác suất này luôn bằng 1. Điều này rất hữu ích trong các bài toán phân loại, vì nó cho phép mô hình dự đoán xác suất của từng lớp.
- **Cân bằng các giá trị đầu ra:** Softmax không chỉ đơn giản chọn ra lớp có xác suất cao nhất mà còn chuẩn hóa các giá trị đầu ra, giúp mô hình phân biệt rõ hơn giữa các lớp.
- **Đặc biệt hữu ích trong phân loại nhiều lớp:** Đối với các bài toán có nhiều hơn hai lớp (multi-class classification), Softmax là lựa chọn lý tưởng cho lớp đầu ra.

2.2.5. Hàm mất mát (Loss Function)

Hàm mất mát (loss function) là một hàm đo lường mức độ khác biệt giữa dự đoán của mô hình và giá trị thực tế (nhãn). Nó giúp mô hình học cách điều chỉnh các trọng số để giảm thiểu sai số trong dự đoán qua quá trình huấn luyện. Trong học sâu (deep learning), hàm mất mát đóng vai trò quan trọng vì nó định hướng quá trình tối ưu hóa (optimization) để cải thiện độ chính xác của mô hình [13].

2.3. Mạng nơ-ron hồi quy (RNN)

2.3.1. Khái niệm

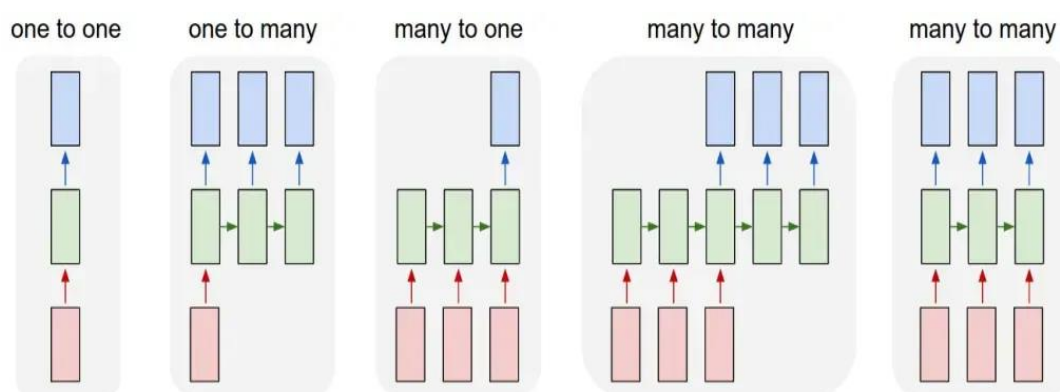
Mạng nơ-ron hồi quy (Recurrent Neural Network - RNN) là một trong những thuật toán nổi bật trong lĩnh vực xử lý ngôn ngữ tự nhiên. Khác với các mô hình mạng nơ-ron truyền thống, trong đó đầu vào và đầu ra được xử lý độc lập, RNN có khả năng xử lý chuỗi dữ liệu và tạo ra đầu ra dựa trên cả các đầu vào hiện tại và những thông tin từ các bước tính toán trước đó [13].

2.3.2. Đặc điểm

- **Khả năng ghi nhớ:** RNN có cấu trúc cho phép nó ghi nhớ thông tin trong quá trình tính toán, điều này rất hữu ích khi làm việc với dữ liệu chuỗi như văn bản hoặc tín hiệu thời gian. Mỗi đầu ra không chỉ phụ thuộc vào đầu vào hiện tại mà còn dựa vào trạng thái ẩn từ bước trước, tạo ra một mối liên kết giữa các bước trong chuỗi [12].
- **Ứng dụng đa dạng:** RNN được áp dụng rộng rãi trong nhiều lĩnh vực, từ dịch máy, phân tích cảm xúc, cho đến dự đoán chuỗi thời gian. Nhờ khả năng tiếp cận ngữ cảnh, RNN có thể hiểu và xử lý các mối quan hệ trong dữ liệu tuần tự, như trong câu văn hoặc dòng thời gian của các sự kiện [12].
- **Cấu trúc linh hoạt:** RNN có thể được mở rộng thành các biến thể như Long Short-Term Memory (LSTM) hoặc Gated Recurrent Unit (GRU), giúp cải thiện khả năng xử lý thông tin dài hạn và giảm thiểu vấn đề "gradient vanishing", một vấn đề phổ biến trong huấn luyện mạng nơ-ron sâu [12].

2.3.3. Các dạng bài toán của RNN

- **One to one:** Chỉ có một input kết nối với một output duy nhất, chẳng hạn như các bài toán phân loại hình ảnh [12].
- **One to many:** Một input liên kết với nhiều chuỗi output, phổ biến là các bài toán đặt caption cho ảnh [12].
- **Many to One:** Nhiều input nhưng chỉ có output, ví dụ phổ biến là bài toán phân loại cảm xúc [12].
- **Many to many:** Nhiều input và nhiều output, chẳng hạn như phân loại video [12].



Hình 2.4 Các dạng toán của mô hình RNN

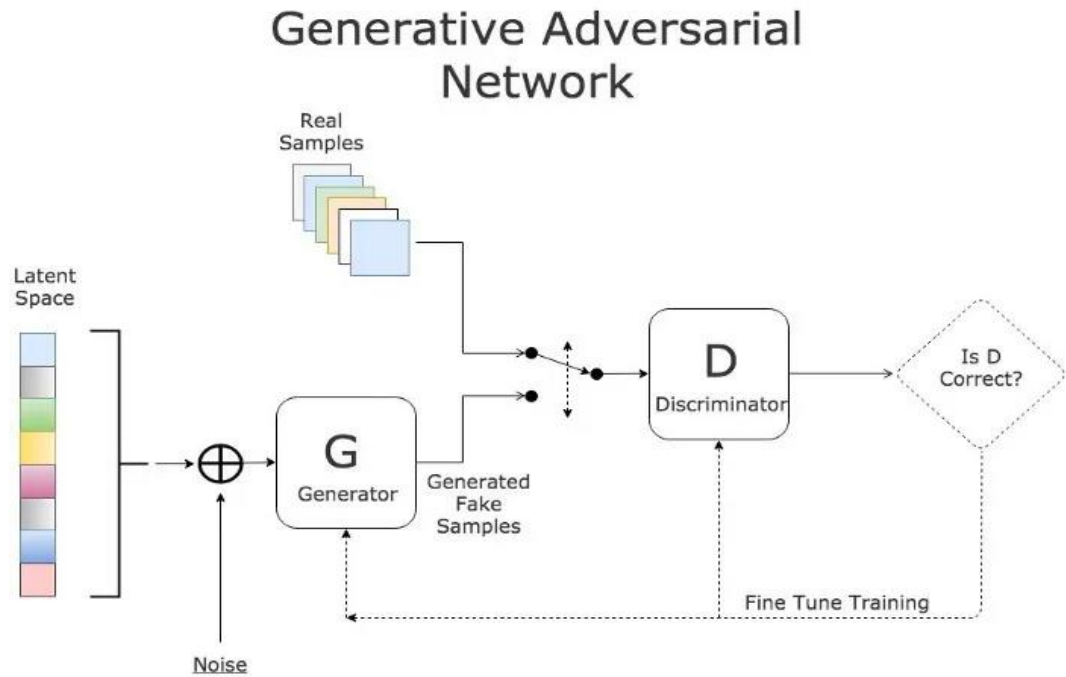
2.3.4. Mạng sinh đối nghịch (GAN)

Mạng đối kháng sinh (Generative Adversarial Networks - GAN) là một lớp mô hình mạnh mẽ được thiết kế để tạo ra dữ liệu giả tương tự như dữ liệu thật. Tên gọi của mạng này xuất phát từ cấu trúc gồm hai mạng có vai trò đối kháng lẫn nhau: **Generator** và **Discriminator** [10] [13] [14].

Cấu trúc và hoạt động của GAN:

- **Generator** [10] [12]: Mạng này có nhiệm vụ sinh ra dữ liệu giả với mục tiêu đánh lừa mạng Discriminator. Nó học cách tạo ra các mẫu mà có thể khiến Discriminator không phân biệt được đâu là dữ liệu thật, đâu là dữ liệu giả.

- **Discriminator** [10] [12]: Mạng này chịu trách nhiệm phân loại dữ liệu đầu vào thành hai loại: dữ liệu thật và dữ liệu giả. Nó học cách nhận diện các đặc điểm của dữ liệu thật và cải thiện khả năng phân loại của mình thông qua việc nhận diện những mẫu do Generator tạo ra.



Hình 2.5 Kiến trúc của GAN

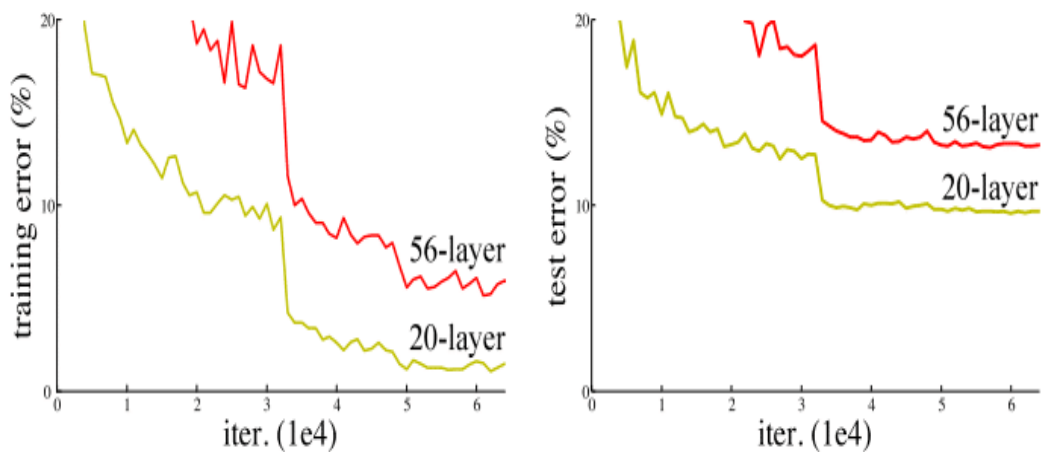
Một số ứng dụng phổ biến của GAN là: Tạo khuôn mặt người, thay đổi độ tuổi khuôn mặt, sinh ảnh vật thể, tạo nhân vật hoạt hình,...

CHƯƠNG 3. KIẾN TRÚC MẠNG NƠ-RON ResNeXt101 VÀ LSTM

3.1. Mô hình ResNet (Residual Networks)

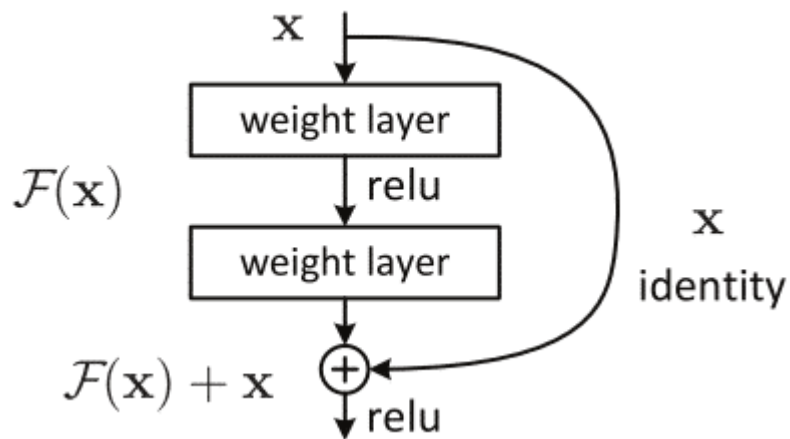
Mô hình **ResNet** (Residual Networks) đã đạt được sự chú ý rộng rãi sau cuộc thi LSVRC2012 và trở thành một kiến trúc mạng nơ-ron sâu quan trọng trong lĩnh vực **thị giác máy**. Nó đã giải quyết hiệu quả vấn đề huấn luyện các mạng có độ sâu lớn (hàng trăm hoặc hàng nghìn lớp), vốn trước đây rất khó khăn do vấn đề **vanishing gradient** (mất dần độ dốc) [15].

Vanishing gradient: Khi mạng nơ-ron trở nên quá sâu, gradient truyền ngược trong quá trình huấn luyện sẽ bị giảm dần do phép nhân lặp đi lặp lại của các gradient, dẫn đến việc gradient trở nên quá nhỏ, làm mạng không học được gì hoặc học rất chậm.. [15]



Hình 3.1 Mối tương quan giữa độ sâu và hiệu suất mạng

ResNet giải quyết vấn đề này bằng cách sử dụng **skip connections** (kết nối bỏ qua) giữa các lớp, cho phép gradient được truyền trực tiếp đến các lớp trước đó mà không bị giảm quá nhiều. Điều này giúp mô hình có thể học tốt hơn ngay cả khi độ sâu tăng lên.



Hình 3.2 ResNet sử dụng kết nối tắt xuyên qua một hay nhiều lớp

Sau khi **ResNet** được giới thiệu và đạt thành công, nhiều biến thể và cải tiến của kiến trúc này đã xuất hiện nhằm giải quyết các vấn đề mới, hoặc để mở rộng khả năng của ResNet trong các bài toán khác nhau như **ResNeXt**, **DenseNet**, **Wide ResNet**, **Squeeze-and-Excitation Networks (SENet)** là những biến thể nổi bật của ResNet. Chúng đã được đề xuất nhằm cải thiện hiệu quả của mạng nơ-ron trong việc huấn luyện với độ sâu lớn hơn hoặc trong các ứng dụng phức tạp hơn.

3.2. Mô hình ResNext CNN

ResNeXt là một mô hình mạng phiên bản nâng cấp của **ResNet**. ResNeXt vẫn giữ lại đặc trưng **skip connections** (các kết nối tắt giữa các lớp) của ResNet để giúp giảm thiểu vấn đề mất mát gradient trong các mạng sâu. Tuy nhiên, ResNeXt sẽ bổ sung thêm một đặc tính mới là **grouped convolution**. Grouped convolution cho phép chia đầu vào thành nhiều nhóm nhỏ và thực hiện tích chập trên từng nhóm riêng biệt, giúp tối ưu hóa khả năng học và hiệu suất tính toán của mạng [16].

3.3. Mô hình ResNext CNN ResNext50

Hiện nay để nhận diện Deepfake bằng các mô hình mạng nơ-ron CNN kết hợp với mô hình mạng nơ-ron hồi LSTM thì ta có thể dễ dàng tìm kiếm được các dự án sử dụng mô hình ResNext50_32x4d.

stage	output	ResNeXt-50 (32×4d)
conv1	112×112	7×7, 64, stride 2
		3×3 max pool, stride 2
conv2	56×56	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128, C=32 \\ 1 \times 1, 256 \end{bmatrix} \times 3$
conv3	28×28	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256, C=32 \\ 1 \times 1, 512 \end{bmatrix} \times 4$
conv4	14×14	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512, C=32 \\ 1 \times 1, 1024 \end{bmatrix} \times 6$
conv5	7×7	$\begin{bmatrix} 1 \times 1, 1024 \\ 3 \times 3, 1024, C=32 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$
	1×1	global average pool 1000-d fc, softmax
# params.		25.0×10⁶

Hình 3.3 Kiến trúc mô hình ResNext50

Sơ đồ này mô tả chi tiết các tầng tích chập trong **ResNeXt50_32x4d**, từ tầng tích chập đầu tiên đến các tầng residual với grouped convolution. Mô hình sử dụng grouped convolution để tối ưu hóa tính toán mà vẫn duy trì khả năng học các đặc trưng phức tạp.

Lợi ích của ResNeXt50:

- **Hiệu suất cao hơn:** So với ResNet50, ResNeXt50 có thể đạt hiệu suất cao hơn trong các bài toán về nhận dạng hình ảnh, phân loại và các tác vụ thị giác máy tính khác nhờ sử dụng grouped convolution.
- **Tối ưu hóa tính toán:** Sử dụng grouped convolution giúp giảm độ phức tạp tính toán và số lượng tham số mà không ảnh hưởng đến khả năng học của mạng.
- **Tính linh hoạt:** ResNeXt có thể dễ dàng mở rộng bằng cách thay đổi số nhóm hoặc số chiều kênh trong các tầng tích chập mà không cần tăng quá nhiều tham số hoặc độ phức tạp.

3.4. Mô hình ResNext CNN ResNext101

Mô hình ResNeXt CNN: Trong đề tài này, em sử dụng mô hình Residual Convolutional Neural Network đã được huấn luyện trước. Tên của mô hình là **resnext101_32x8d** để so sánh với phiên bản **resnext50_32x4d**.

ResNeXt101_32x8d [17] là một biến thể của kiến trúc ResNeXt, được thiết kế để cải thiện hiệu suất của mạng nơ-ron tích chập bằng cách sử dụng các phép tích chập nhóm. Bao gồm 101 lớp (layers) và cấu hình 32x8d là có 32 nhóm (group) được sử dụng trong các phép tích chập nhóm và **8d** là số lượng kênh (channel) trong mỗi nhóm..

PyTorch cung cấp sẵn các mô hình ResNeXt101_32x8d đã được huấn luyện trước, vì ta có thể dễ dàng tải và sử dụng trong dự án của mình [17]

```
import torch
import torchvision.models as models

model = models.resnext101_32x8d(pretrained=True)
model.eval()

# Kiểm tra kiến trúc mô hình
print(model)
```

Hình 3.4 Tải mô hình ResNeXt101_32x8d đã được huấn luyện trước

Sequential Layer là một lớp trong PyTorch thuộc mô-đun `torch.nn.Sequential`, được sử dụng để xây dựng mô hình mạng nơ-ron dưới dạng chuỗi (tuần tự). Nó cho phép ta có thể sắp xếp các lớp (layers) theo thứ tự, mỗi lớp sẽ nhận đầu vào từ lớp trước đó và đưa đầu ra cho lớp kế tiếp.

Khi kết hợp với ResNeXt và LSTM nó đóng vai trò lưu trữ và xử lý vector đặc trưng này theo cách có thứ tự, trước khi chuyển nó sang **LSTM** (Long Short-Term Memory), mô hình có khả năng xử lý dữ liệu theo chuỗi.

Cách hoạt động :

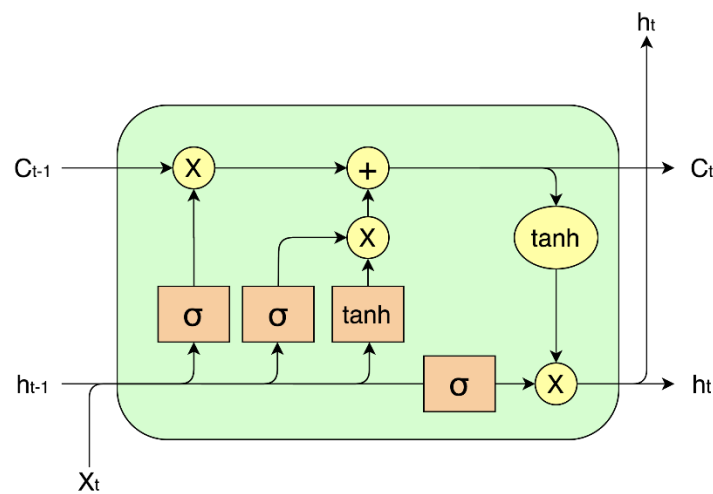
- ResNeXt xử lý hình ảnh đầu vào và trích xuất ra các đặc trưng dưới dạng vector.
- Các đặc trưng này sau đó được sắp xếp trong **Sequential Layer**.

- Sau đó, các đặc trưng được truyền tuần tự vào **LSTM** để dự đoán dựa trên thông tin thời gian hoặc chuỗi.

3.5. Mạng nơ-ron hồi quy LSTM trong việc xử lý chuỗi

3.5.1. Khái niệm

LSTM (Long Short-Term Memory)[5] là một loại mạng RNN đặc biệt, được thiết kế để xử lý các dữ liệu tuần tự, như chuỗi thời gian hoặc video. Nó có khả năng nhớ thông tin qua nhiều bước thời gian (temporal memory), nên rất phù hợp cho các bài toán phân tích dữ liệu tuần tự.



Hình 3.5 Kiến trúc tổng quan của LSTM

3.5.2. Cấu trúc của một tầng LSTM

Cấu trúc của một tầng LSTM bao gồm [13] :

- Cổng quên (forget gate) quyết định loại thông tin nào cần loại bỏ khỏi trạng thái nhớ.
- Cổng đầu vào (input gate) quyết định thông tin nào sẽ được thêm vào trạng thái nhớ.
- Cổng đầu ra (output gate) quyết định phần nào của trạng thái nhớ sẽ được sử dụng làm đầu ra cho bước thời gian hiện tại.

3.5.3. Quá trình xử lý trong LSTM

3.5.3.1. Cơ chế truyền thông tin giữa các bước thời gian [13]

Ở mỗi bước thời gian (ví dụ: từ thời điểm t đến $t+1$), mạng LSTM nhận một đầu vào mới, và nó sẽ quyết định cách xử lý thông tin dựa trên đầu vào này kết hợp với thông tin từ bước thời gian trước đó.

Đầu vào của mỗi bước bao gồm:

- **Trạng thái ẩn (hidden state)** từ bước trước (đại diện cho thông tin đã được mạng học từ các bước thời gian trước).
- **Trạng thái nhớ (cell state)**, đó là nơi lưu trữ thông tin dài hạn.

3.5.3.2. Cách LSTM học cách duy trì hoặc loại bỏ thông tin qua các cổng điều khiển

Cổng quên (Forget Gate):

- Cổng này quyết định phần nào của trạng thái nhớ cần được giữ lại và phần nào sẽ bị loại bỏ. Nó xem xét trạng thái ẩn từ bước trước và đầu vào hiện tại, sau đó trả về một giá trị từ 0 đến 1 cho mỗi phần tử của trạng thái nhớ.
- Nếu giá trị là **0**, phần thông tin tương ứng sẽ bị loại bỏ. Nếu giá trị là **1**, phần thông tin đó sẽ được giữ nguyên. Điều này giúp mạng quên đi những thông tin không còn cần thiết và giữ lại những thông tin quan trọng từ các bước thời gian trước.

Cổng đầu vào (Input Gate):

- Cổng này kiểm soát việc cập nhật thông tin mới vào trạng thái nhớ. Đầu tiên, nó quyết định phần nào của thông tin mới từ đầu vào cần được thêm vào trạng thái nhớ.
- Một mạng nơ-ron sẽ quyết định những phần nào của đầu vào hiện tại và trạng thái ẩn trước đó cần được lưu vào trạng thái nhớ mới. Sau đó, trạng thái nhớ hiện tại sẽ được cập nhật với những thông tin mới quan trọng.

Trạng thái nhớ (Cell State):

- Trạng thái nhớ là một vector đặc biệt mà thông tin dài hạn được lưu trữ. Nó là nơi mà LSTM lưu giữ thông tin từ các bước thời gian trước để sử dụng ở các bước thời gian sau.
- Khi trạng thái nhớ được cập nhật qua mỗi bước, nó sẽ kết hợp các thông tin mới thông qua cổng đầu vào và loại bỏ những thông tin không cần thiết qua cổng quên.

Cổng đầu ra (Output Gate):

- Cổng này quyết định phần nào của trạng thái nhớ sẽ được sử dụng làm đầu ra tại bước thời gian hiện tại. Nó sẽ tính toán một giá trị dựa trên đầu vào hiện tại và trạng thái ẩn từ bước trước.
- Phần thông tin được chọn sẽ trở thành trạng thái ẩn mới (hidden state), sau đó sẽ được chuyển đến bước thời gian tiếp theo và được sử dụng để tiếp tục xử lý chuỗi thời gian.

3.5.3.3. LSTM có thể xử lý dữ liệu dài hạn nhờ cơ chế nhớ (Cell State)

Trạng thái nhớ (Cell State) là yếu tố quan trọng giúp LSTM vượt qua hạn chế của các mạng nơ-ron hồi quy truyền thống (RNN). Nó cho phép thông tin được truyền dọc theo chuỗi thời gian mà không bị mất đi hoặc giảm hiệu quả sau nhiều bước thời gian.

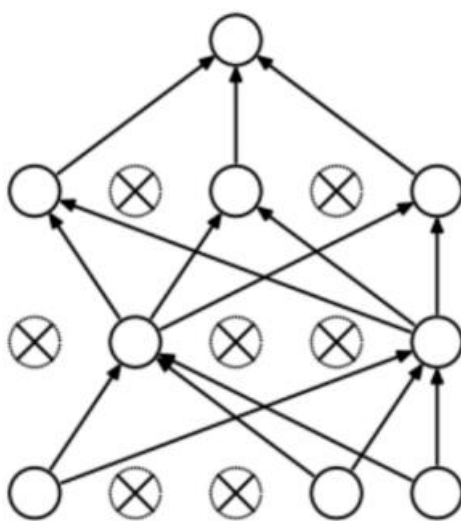
Nhờ cơ chế này, **LSTM có khả năng lưu trữ thông tin quan trọng trong thời gian dài**, cho phép mô hình dự đoán không chỉ dựa trên dữ liệu gần đây mà còn dựa trên thông tin từ các bước thời gian xa hơn. Điều này rất hữu ích trong các bài toán xử lý video hoặc chuỗi dữ liệu phức tạp, nơi mà mối quan hệ giữa các bước thời gian cách xa nhau cần được ghi nhớ và khai thác.

3.5.4. Dropout trong mạng LSTM

Dropout Layer là một kỹ thuật phổ biến được sử dụng trong huấn luyện mạng nơ-ron để giảm **overfitting** (quá khớp). **Dropout** được giới thiệu bởi Geoffrey Hinton và các cộng sự vào năm 2012, và nó hoạt động bằng cách tạm thời bỏ qua (bỏ qua ngẫu nhiên) một số **neuron** trong quá trình huấn luyện mô hình. Điều này giúp

mạng học được các đặc trưng mạnh mẽ và tổng quát hơn, thay vì chỉ ghi nhớ các mẫu cụ thể trong tập huấn luyện.

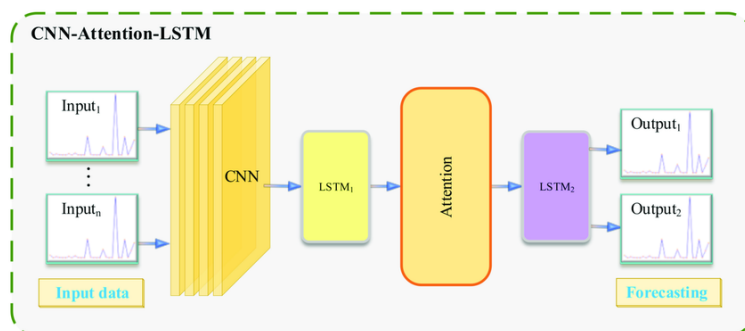
Trong mô hình, một tầng **Dropout** với giá trị **0.4** được sử dụng nhằm ngăn chặn tình trạng **overfitting** (quá khớp). Kỹ thuật Dropout giúp mô hình tổng quát hóa tốt hơn bằng cách ngẫu nhiên đặt giá trị đầu ra của một số neuron thành **0**. Khi các neuron bị vô hiệu hóa như vậy, hàm mất mát (cost function) trở nên nhạy cảm hơn với các thay đổi từ các neuron lân cận, từ đó làm thay đổi cách các trọng số (weights) sẽ được cập nhật trong quá trình lan truyền ngược (backpropagation).



Hình 3.6 Tổng quan tầng **Dropout**

3.5.5. Tầng chú ý(Attention Layer) [18]

Tầng chú ý (Attention Layer) là một thành phần quan trọng trong nhiều mô hình học sâu, đặc biệt là trong các nhiệm vụ xử lý chuỗi như phân loại video, xử lý ngôn ngữ tự nhiên và nhiều ứng dụng khác. Tầng chú ý giúp mô hình tập trung vào các phần cụ thể của đầu vào mà nó cho là quan trọng hơn trong quá trình ra quyết định.



Hình 3.7 Attention Layer trong mô hình CNN và LSTM

Cách hoạt động của Attention Layer:

- **Tính toán Trọng số Chú ý:** Trong phương thức forward, đầu vào là đầu ra từ LSTM (lstm_output). Mạng nơ-ron chú ý sẽ tính toán trọng số chú ý cho từng khung hình trong chuỗi.
- **Xây dựng Vector Ngữ cảnh:** Các trọng số chú ý sau đó được sử dụng để tính toán một vector ngữ cảnh bằng cách nhân trọng số với đầu ra từ LSTM và tổng hợp chúng. Vector ngữ cảnh này sẽ đại diện cho thông tin quan trọng nhất từ chuỗi video mà mô hình cần sử dụng để ra quyết định cuối cùng

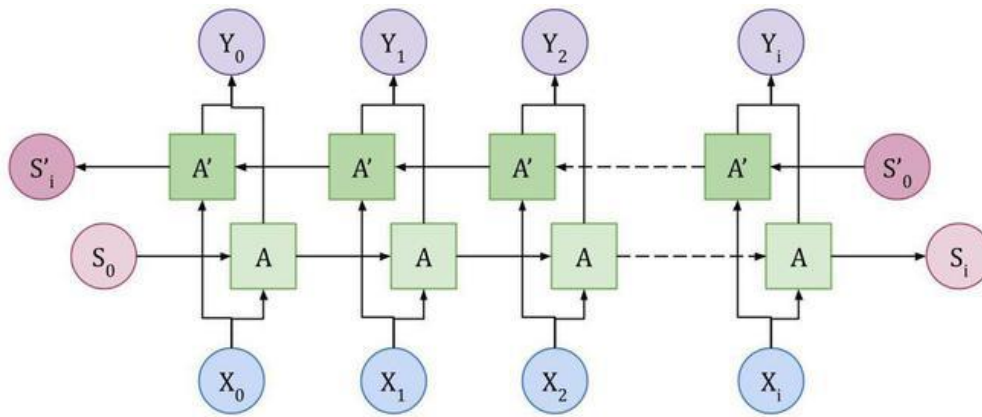
3.6. LSTM 2 chiều (Bidirectional LSTM)

Bidirectional LSTM (BiLSTM) là một biến thể của mô hình LSTM (Long Short-Term Memory), một loại mạng nơ-ron hồi quy (RNN) mạnh mẽ trong việc xử lý chuỗi dữ liệu. BiLSTM cho phép mô hình học từ cả hai chiều của dữ liệu chuỗi thời gian, thay vì chỉ một chiều như LSTM truyền thống.

3.6.1. Kiến trúc của BiLSTM

Kiến trúc của BiLSTM:

- BiLSTM bao gồm hai LSTM, một chạy theo chiều tiến (forward) và một chạy theo chiều lùi (backward).
- Đầu ra của hai LSTM này sẽ được kết hợp (thường bằng cách nối lại) để tạo ra đầu ra cuối cùng cho mỗi thời điểm trong chuỗi.



Hình 3.8 Kiến trúc của Bidirectional LSTM

Thành phần trong sơ đồ:

- $X_0, X_1, X_2, \dots, X_i$: Đây là đầu vào của chuỗi dữ liệu, tương ứng với từng thời điểm trong chuỗi (các vector đầu vào theo thời gian).
- $Y_0, Y_1, Y_2, \dots, Y_i$: Đây là các đầu ra của mô hình tại từng thời điểm tương ứng.
- S_0, S_1 và S'_0, S'_1 : Các trạng thái ẩn (hidden states) của LSTM trong cả hai chiều:
 - + S : Trạng thái ẩn của LSTM chiều thuận (forward LSTM).
 - + S' : Trạng thái ẩn của LSTM chiều ngược (backward LSTM).

Giải thích kiến trúc:

- **Forward LSTM (A)**: Mạng LSTM tiến lên (các khối màu xanh lá trong sơ đồ) nhận dữ liệu từ quá khứ đến hiện tại, xử lý chuỗi từ trái sang phải: $X_0 \rightarrow X_1 \rightarrow X_2 \rightarrow \dots \rightarrow X_i$
- **Backward LSTM (A')**: Mạng LSTM ngược (các khối A' màu xanh) chạy theo chiều ngược lại, xử lý dữ liệu từ hiện tại về quá khứ: $X_0 \rightarrow X_1 \rightarrow X_2 \rightarrow \dots \rightarrow X_i$
- **Kết hợp trạng thái ẩn**: Ở mỗi thời điểm, đầu ra của hai mạng LSTM (cả tiến và lùi) được kết hợp lại (thường là nối chuỗi hoặc lấy trung bình) để tạo ra đầu ra cuối cùng Y .

3.6.2. Ưu điểm của BiLSTM

Khai thác tốt hơn ngữ cảnh: Trong các bài toán như xử lý ngôn ngữ tự nhiên (NLP), thông tin sau một từ có thể quan trọng như thông tin trước đó. BiLSTM cho phép mô hình tận dụng cả hai chiều thông tin này.

Cải thiện hiệu suất: BiLSTM thường cho kết quả tốt hơn trong các tác vụ như nhận dạng thực thể, phân loại chuỗi, dịch máy (machine translation), vì nó có khả năng "nhìn" cả phía trước và phía sau của chuỗi dữ liệu.

3.6.3. Ứng dụng

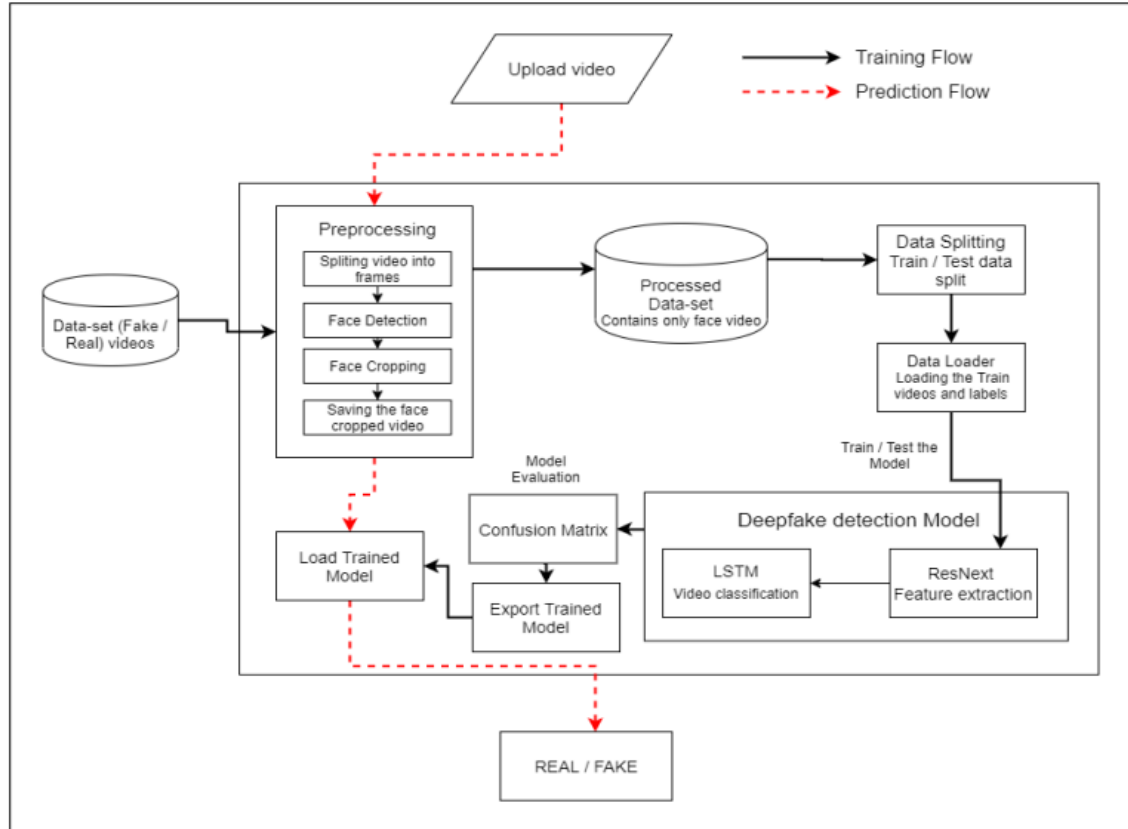
Xử lý ngôn ngữ tự nhiên (NLP): BiLSTM rất phổ biến trong các bài toán phân tích văn bản như gán nhãn từ, dịch máy, nhận dạng thực thể có tên (NER), phân loại văn bản.

Nhận dạng giọng nói: BiLSTM có thể học được cả ngữ cảnh trước và sau của một từ trong chuỗi âm thanh, giúp nhận dạng tốt hơn.

Phân tích chuỗi thời gian: BiLSTM được dùng trong các bài toán dự đoán tài chính, phát hiện bất thường, và các ứng dụng liên quan đến chuỗi dữ liệu khác.

CHƯƠNG 4. XÂY DỰNG HỆ THỐNG

4.1. Tổng quan về hệ thống



Hình 4.1 Sơ đồ của hệ thống

Luồng dữ liệu:

- Có hai luồng chính: Luồng huấn luyện (mũi tên liền) và Luồng dự đoán (mũi tên đứt).

Đầu vào:

- Dataset chứa cả video thật và giả (fake/deepfake).
- Khả năng tải lên video mới để dự đoán.

Tiền xử lý:

- Chia video thành các khung hình (frames).
- Phát hiện khuôn mặt trong mỗi khung.
- Cắt (crop) khuôn mặt đã phát hiện.
- Lưu video đã cắt khuôn mặt.

Xử lý dữ liệu:

- Tạo bộ dữ liệu đã xử lý chỉ chứa video khuôn mặt.
- Chia dữ liệu thành tập huấn luyện và kiểm tra.
- Sử dụng Data Loader để nạp dữ liệu và nhãn.

Mô hình phát hiện Deepfake:

- Sử dụng ResNext để trích xuất đặc trưng từ video.
- Áp dụng LSTM để phân loại video dựa trên các đặc trưng đã trích xuất.

Huấn luyện và đánh giá:

- Huấn luyện mô hình trên dữ liệu đã chuẩn bị.
- Đánh giá mô hình sử dụng ma trận nhầm lẫn (confusion matrix).
- Xuất mô hình đã huấn luyện để sử dụng sau này.

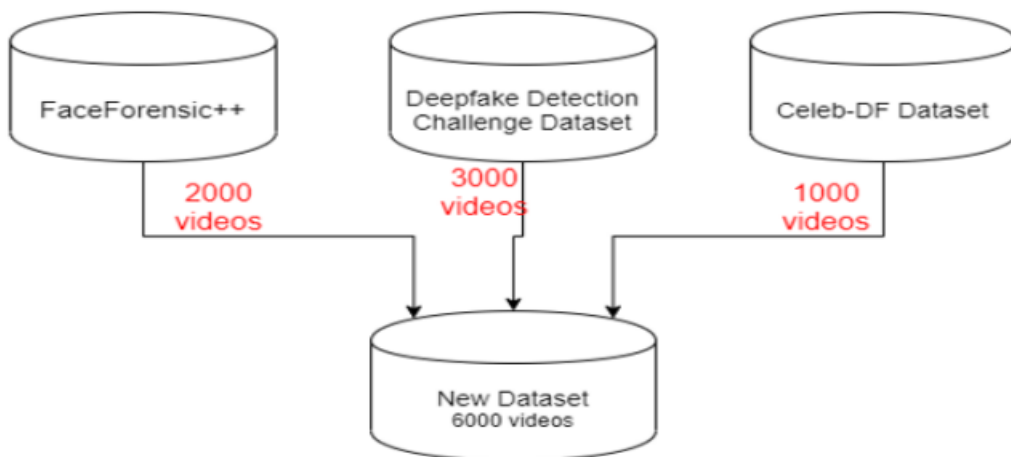
Dự đoán:

- Khi có video mới được tải lên, nó sẽ đi qua quá trình tiền xử lý.
- Sử dụng mô hình đã huấn luyện để dự đoán.
- Kết quả cuối cùng là phân loại video là THẬT hoặc GIẢ.

4.2. Chuẩn bị dữ liệu

Để tập dữ liệu được phong em có thu tập từ các tập dữ liệu có sẵn khác nhau như FaceForensic++(FF), Deepfake detection challenge(DFDC), and Celeb-DF, sau đó trộn các dữ liệu đó thành một tập dữ liệu mới bao gồm một tập chứa 6000 video và một file csv chứa lable (thật, giả)của video trong tập dữ liệu , để tối ưu cấu hình trong quá trình làm đề tài của tập dữ liệu thì cần vài bước tiền xử lý dữ liệu cần thiết video trong tập dữ liệu.

Dữ liệu thu tập được em sẽ phân bổ như sau:



Hình 4.2 Tập dữ liệu thu thập được

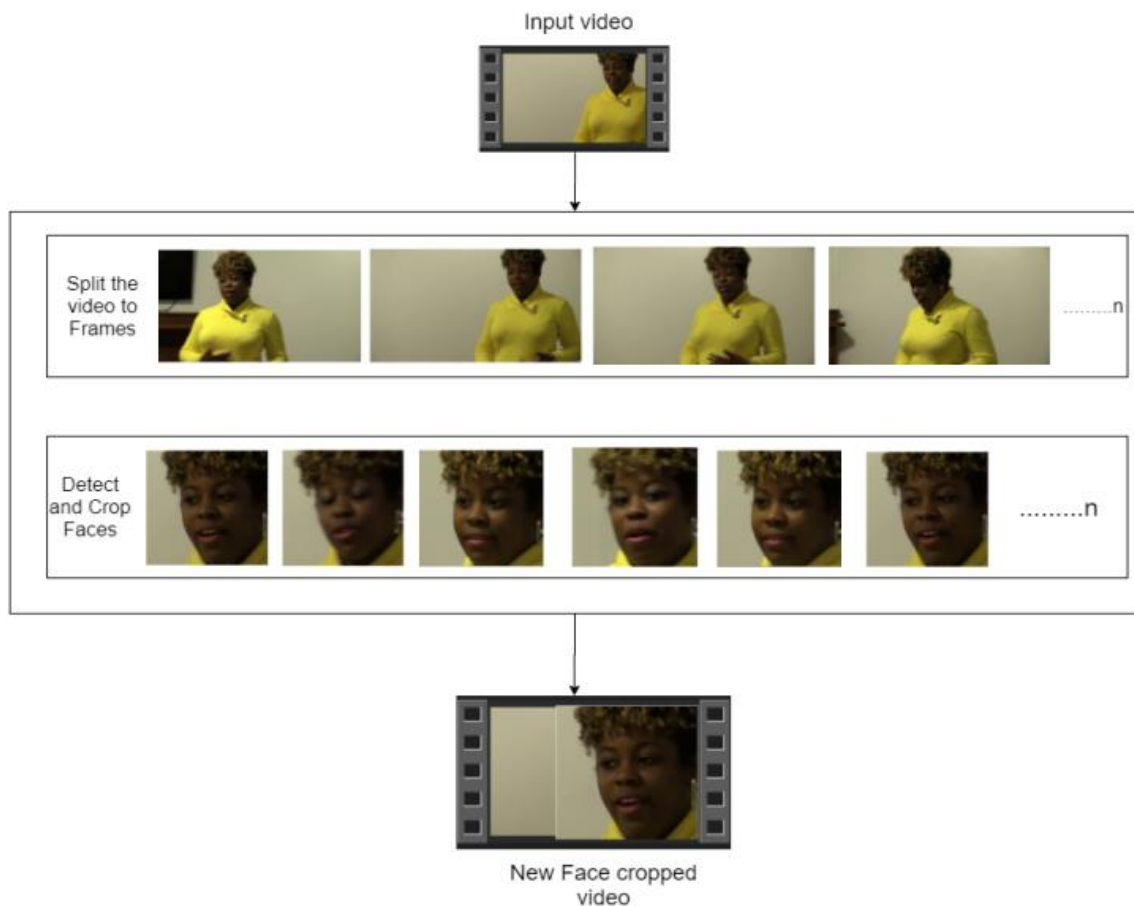
4.3. Tiền xử lý dữ liệu

Đầu tiên trong quá trình tiền xử lý video là loại bỏ tất cả các phần không cần thiết và nhiễu, chỉ giữ lại các đoạn có khuôn mặt vì vậy em sẽ làm theo các bước dưới đây

- **Phân tách video thành khung hình:** Video gốc được chia thành từng khung hình riêng biệt, tạo điều kiện thuận lợi cho việc phát hiện khuôn mặt.
- **Phát hiện khuôn mặt trong từng khung hình:** Mỗi khung hình sẽ được kiểm tra để xác định xem có khuôn mặt hay không. Những khung hình không chứa khuôn mặt sẽ bị loại bỏ.
- **Cắt khung hình theo khuôn mặt:** Các khung hình chứa khuôn mặt sẽ được cắt để chỉ giữ lại phần có khuôn mặt.
- **Kết hợp khung hình để tạo video mới:** Các khung hình đã cắt sẽ được ghép lại với nhau để tạo thành một video mới, chỉ bao gồm các khung hình có khuôn mặt.
- **Loại bỏ khung hình không có khuôn mặt:** Trong quá trình xử lý, những khung hình không phát hiện khuôn mặt bị bỏ qua.
- **Giới hạn số khung hình do khả năng tính toán:** Để đồng nhất thời lượng video và khung hình trong các video nên em chọn ra ngưỡng giá trị thời

lượng và khung hình dựa trên giá trị trung bình . Giá trị được chọn là thời lượng 10 giây và khung hình là 150 khung hình.

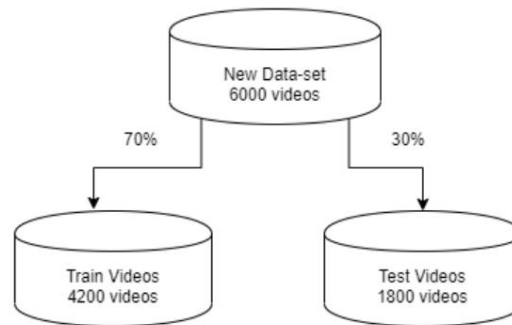
- **Lưu trữ khung hình trong tập dữ liệu mới:** Khi tạo ra tập dữ liệu mới, chúng tôi chỉ lưu lại 150 khung hình đầu tiên của video.
- **Xử lý tuần tự với LSTM:** Để thể hiện hiệu quả của Bộ nhớ dài hạn ngắn hạn (LSTM), chúng tôi sẽ xử lý các khung hình theo thứ tự, tức là chỉ sử dụng 150 khung hình đầu tiên mà không chọn ngẫu nhiên.
- **Cấu hình video mới:** Video đã được xử lý sẽ được lưu với tốc độ khung hình 30 fps và độ phân giải 112 x 112.



Hình 4.3 Tiền xử lý dữ liệu

4.4. Phân chia tập dữ liệu

Sau khi tiền xử lý tập dữ liệu xong thì em sẽ chia tập dữ liệu với tỷ lệ 7:3 trong đó 70% (Khoảng 4200 video) của tập dữ liệu dùng cho việc huấn luyện học máy sâu 30% (khoảng 1800 video) dành cho việc thử nghiệm với mô hình.



Hình 4.4 Phân chia dữ liệu

4.5. Xây Dựng kiến trúc mô hình

Ngoài việc sử dụng kiến trúc mạng nơ ron CNN và RNN em có kết hợp thêm một số kỹ thuật để tối ưu độ chính xác cho mô hình.

4.5.1. Import các thư viện cần thiết:

Mã nguồn:

```
1 import torch
2 import torchvision
3 from torchvision import transforms
4 from torch.utils.data import DataLoader
5 from torch.utils.data.dataset import Dataset
6 import os
7 import numpy as np
8 import cv2
9 import matplotlib.pyplot as plt
10 import face_recognition
```

Đây là đoạn code dùng để khai báo các thư viện cần thiết trong quá trình chạy chương trình

4.5.2. Định nghĩa lớp AttentionLayer:

Mã nguồn :

```
1 class HighAccuracyVideoClassifier(nn.Module):
2     def __init__(
3         self,
4         num_classes,
5         latent_dim=2048,
6         lstm_layers=2,
7         hidden_dim=1024,
8         bidirectional=True,
9         dropout=0.5,
10    ):
11        super(HighAccuracyVideoClassifier, self).__init__()
12        # Load pretrained model
13        self.backbone = models.resnext101_32x8d(pretrained=True)
14        # Remove the last two layers (avgpool and fc)
15        self.features =
16 nn.Sequential(*list(self.backbone.children())[:-2])
17
18        # Fine-tune the last two layers of the backbone
19        for param in list(self.features.parameters())[:-2]:
20            param.requires_grad = False
21
22        self.avgpool = nn.AdaptiveAvgPool2d(1)
23        self.lstm = nn.LSTM(
24            latent_dim,
25            hidden_dim,
26            lstm_layers,
27            bidirectional=bidirectional,
28            batch_first=True,
29            dropout=dropout,
30        )
31
32        self.attention = AttentionLayer(hidden_dim * 2 if
33 bidirectional else hidden_dim)
34
35        fc_in_dim = hidden_dim * 2 if bidirectional else hidden_dim
36        self.fc = nn.Sequential(
37            nn.Linear(fc_in_dim, 1024),
```

```

38         nn.ReLU(),
39         nn.Dropout(dropout),
40         nn.Linear(1024, 512),
41         nn.ReLU(),
42         nn.Dropout(dropout),
43         nn.Linear(512, num_classes),
44     )
45     self.fc_aux = nn.Linear(latent_dim, num_classes) # Auxiliary
46 classifier

```

Lớp AttentionLayer này tạo ra một cơ chế attention để tập trung vào các phần quan trọng của đầu ra LSTM. Nó sử dụng hai lớp linear với một hàm kích hoạt tanh ở giữa để tính toán trọng số attention. Sau đó, nó áp dụng softmax để chuẩn hóa trọng số và tính toán vector ngữ cảnh bằng cách nhân trọng số với đầu ra LSTM.

4.5.3. Định nghĩa lớp HighAccuracyVideoClassifier:

Mã nguồn :

```

1 class HighAccuracyVideoClassifier(nn.Module):
2     def __init__(
3         self,
4         num_classes,
5         latent_dim=2048,
6         lstm_layers=2,
7         hidden_dim=1024,
8         bidirectional=True,
9         dropout=0.5,
10    ):
11        super(HighAccuracyVideoClassifier, self).__init__()
12
13        # Load pretrained model
14        self.backbone = models.resnext101_32x8d(pretrained=True)
15        # Remove the last two layers (avgpool and fc)
16        self.features =
17 nn.Sequential(*list(self.backbone.children())[:-2])
18
19        # Fine-tune the last two layers of the backbone
20        for param in list(self.features.parameters())[:-2]:
21            param.requires_grad = False

```

```

22         self.avgpool = nn.AdaptiveAvgPool2d(1)
23
24         self.lstm = nn.LSTM(
25             latent_dim,
26             hidden_dim,
27             lstm_layers,
28             bidirectional=bidirectional,
29             batch_first=True,
30             dropout=dropout,
31         )
32
33         self.attention = AttentionLayer(hidden_dim * 2 if
34 bidirectional else hidden_dim)
35         fc_in_dim = hidden_dim * 2 if bidirectional else hidden_dim
36         self.fc = nn.Sequential(
37             nn.Linear(fc_in_dim, 1024),
38             nn.ReLU(),
39             nn.Dropout(dropout),
40             nn.Linear(1024, 512),
41             nn.ReLU(),
42             nn.Dropout(dropout),
43             nn.Linear(512, num_classes),
44         )
45         self.fc_aux = nn.Linear(latent_dim, num_classes) # Auxiliary
46 classifier

```

Lớp này định nghĩa kiến trúc tổng thể của mô hình phân loại video. Nó sử dụng một mô hình ResNeXt101 đã được huấn luyện trước làm backbone, sau đó là một LSTM hai chiều, một lớp attention, và các lớp fully connected để phân loại. Nó cũng có một bộ phân loại phụ trợ.

4.5.4. Phương thức forward của HighAccuracyVideoClassifier

Mã nguồn :

```

1 def forward(self, x):
2     batch_size, seq_length, c, h, w = x.shape
3     x = x.view(batch_size * seq_length, c, h, w)
4
5     fmap = self.features(x)

```

```

6     pooled = self.avgpool(fmap)
7     x = pooled.view(batch_size, seq_length, -1)
8
9     aux_out = self.fc_aux(torch.mean(x, dim=1)) # Auxiliary output
10
11     lstm_out, _ = self.lstm(x)
12
13     attn_out = self.attention(lstm_out)
14
15     main_out = self.fc(attn_out)
16
17     return fmap, main_out, aux_out

```

Phương thức này định nghĩa luồng dữ liệu qua mô hình. Nó xử lý đầu vào qua backbone, LSTM, lớp attention, và các lớp fully connected để tạo ra đầu ra chính và đầu ra phụ trợ.

4.5.5. Hàm loss tùy chỉnh

Mã nguồn:

```

1 def custom_loss(main_out, aux_out, targets, alpha=0.2):
2     main_loss = F.cross_entropy(main_out, targets)
3     aux_loss = F.cross_entropy(aux_out, targets)
4
5     return main_loss + alpha * aux_loss

```

Hàm loss này kết hợp loss của đầu ra chính và đầu ra phụ trợ, với một hệ số alpha để cân bằng giữa hai loss.

4.5.6. Khởi tạo mô hình và optimizer

Mã nguồn:

```

1
2
3 model = HighAccuracyVideoClassifier(num_classes=10)
4
5 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
6 model = model.to(device)
7

```

```
8 optimizer = torch.optim.AdamW(model.parameters(), lr=1e-4,
    weight_decay=1e-5)
```

Cuối cùng, mô hình được khởi tạo với 10 lớp, chuyển đến GPU nếu có sẵn, và một optimizer AdamW được tạo ra để huấn luyện mô hình.

4.6. Huấn luyện và đánh giá mô hình

4.6.1. Huấn luyện mô hình

Định nghĩa hàm `train_epoch` để huấn luyện mô hình qua từng epoch

Trong mỗi epoch, mô hình được huấn luyện trên các batch dữ liệu, tính toán loss và accuracy, thực hiện backpropagation và cập nhật trọng số.

Sau mỗi epoch, trạng thái của mô hình được lưu lại.

Mã nguồn :

```
1 def train_epoch(epoch, num_epochs, data_loader, model, criterion,
2 optimizer):
3     model.train()
4     losses = AverageMeter()
5     accuracies = AverageMeter()
6     t = []
7     for i, (inputs, targets) in enumerate(data_loader):
8         if torch.cuda.is_available():
9             targets = targets.type(torch.cuda.LongTensor)
10            inputs = inputs.cuda()
11            _, outputs = model(inputs)
12            loss = criterion(outputs, targets.type(torch.cuda.LongTensor))
13            acc = calculate_accuracy(outputs,
14 targets.type(torch.cuda.LongTensor))
15            losses.update(loss.item(), inputs.size(0))
16            accuracies.update(acc, inputs.size(0))
17            optimizer.zero_grad()
18            loss.backward()
19            optimizer.step()
20            sys.stdout.write(
21                "\r[Epoch %d/%d] [Batch %d / %d] [Loss: %f, Acc: %.2f%"
22                % (
```

```

23         epoch,
24         num_epochs,
25         i,
26         len(data_loader),
27         losses.avg,
28         accuracies.avg))
    torch.save(model.state_dict(), '/content/checkpoint.pt')
    return losses.avg, accuracies.avg

```

4.6.2. Kiểm tra mô hình

Sử dụng hàm `test` để đánh giá mô hình trên tập dữ liệu kiểm tra, hàm này sẽ tính toán loss và accuracy trên tập kiểm tra, đồng thời thu thập các dự đoán để sử dụng sau này.

Mã nguồn :

```

1 def test(epoch, model, data_loader, criterion):
2     print("Testing")
3     model.eval()
4     losses = AverageMeter()
5     accuracies = AverageMeter()
6     pred = []
7     true = []
8     count = 0
9     with torch.no_grad():
10         for i, (inputs, targets) in enumerate(data_loader):
11             if torch.cuda.is_available():
12                 targets = targets.cuda().type(torch.cuda.FloatTensor)
13                 inputs = inputs.cuda()
14                 _, outputs = model(inputs)
15                 loss = torch.mean(criterion(outputs,
16 targets.type(torch.cuda.LongTensor)))
17                 acc = calculate_accuracy(outputs,
18 targets.type(torch.cuda.LongTensor))
19                 _, p = torch.max(outputs, 1)
20                 true += (
21                     (targets.type(torch.cuda.LongTensor))
22                     .detach()

```

```

23         .cpu()
24         .numpy()
25         .reshape(len(targets))
26         .tolist()
27     )
28     pred += p.detach().cpu().numpy().reshape(len(p)).tolist()
29     losses.update(loss.item(), inputs.size(0))
30     accuracies.update(acc, inputs.size(0))
31     sys.stdout.write(
32         "\r[Batch %d / %d]  [Loss: %f, Acc: %.2f%%]"
33         % (i, len(data_loader), losses.avg, accuracies.avg)
34     )
35
36     print("\nAccuracy {}".format(accuracies.avg))
37     return true, pred, losses.avg, accuracies.avg

```

4.6.3. Theo dõi lưu trữ metrics và tính toán độ chính xác

Sử dụng lớp `AverageMeter` để theo dõi và tính toán giá trị trung bình của loss và accuracy trong quá trình huấn luyện và kiểm tra. Và hàm `calculate_accuracy` được sử dụng để tính toán độ chính xác của mô hình.

Mã nguồn :

```

1 class AverageMeter(object):
2     """Computes and stores the average and current value"""
3
4     def __init__(self):
5         self.reset()
6
7     def reset(self):
8         self.val = 0
9         self.avg = 0
10        self.sum = 0
11        self.count = 0
12
13    def update(self, val, n=1):
14        self.val = val
15        self.sum += val * n
16        self.count += n

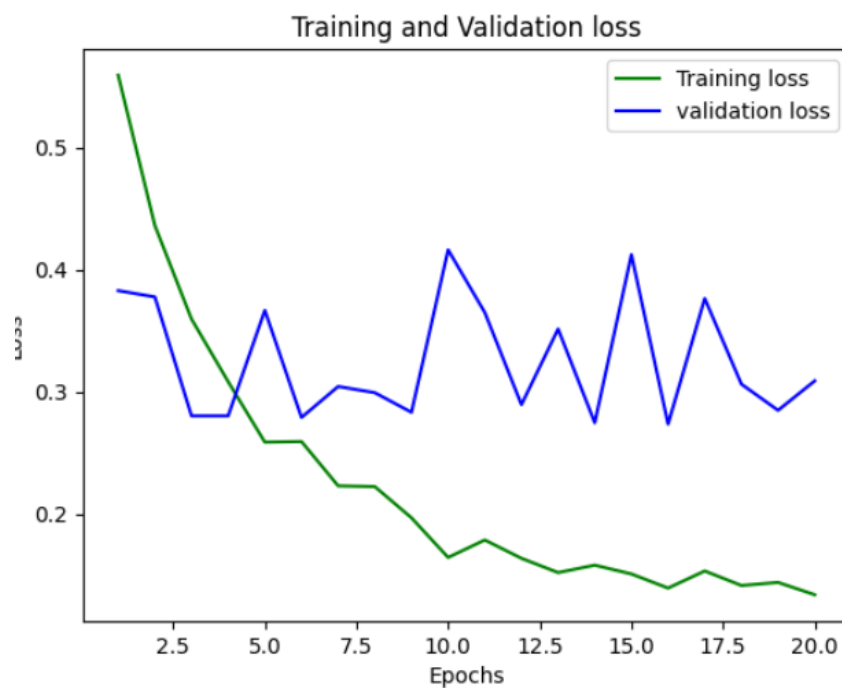
```

```
17         self.avg = self.sum / self.count
18
19
20 def calculate_accuracy(outputs, targets):
21     batch_size = targets.size(0)
22
23     _, pred = outputs.topk(1, 1, True)
24     pred = pred.t()
25     correct = pred.eq(targets.view(1, -1))
26     n_correct_elems = correct.float().sum().item()
27     return 100 * n_correct_elems / batch_size
```

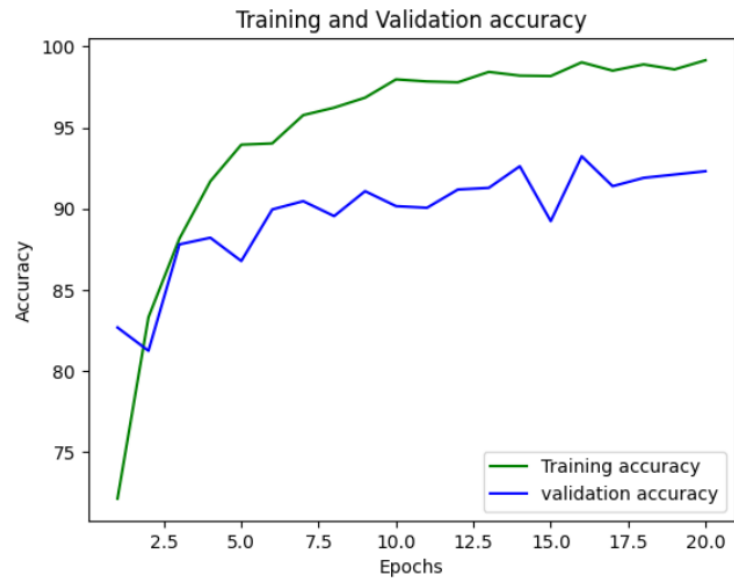
CHƯƠNG 5. KẾT QUẢ ĐẠT ĐƯỢC

5.1. Kết quả của mô hình

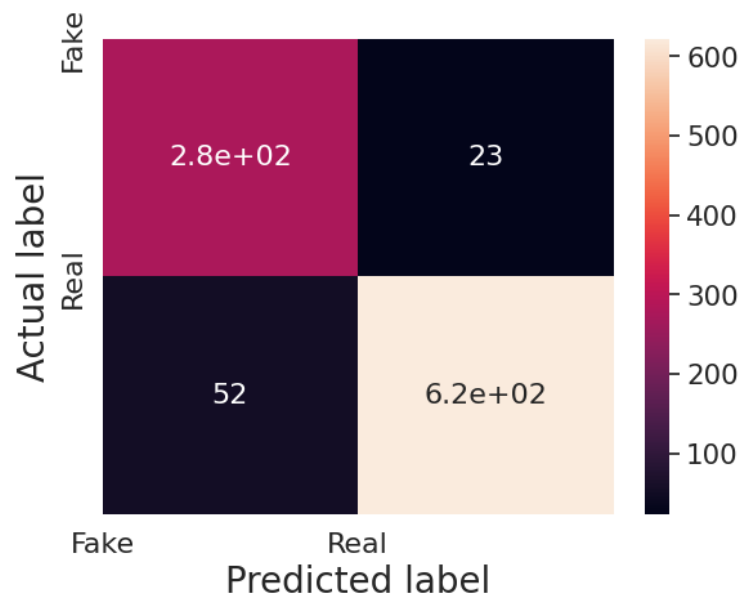
Để huấn luyện mô hình mà có độ chính xác đồng thời có tốc độ xử lý đầu vào em đã huấn luyện thử mô hình với nhiều loại chuỗi khung hình (frame sequence) như 10, 20, 40, 60, 80, 100 trong mô hình của bạn có nhiều mục đích liên quan đến đặc trưng của dữ liệu video và việc phát hiện các thay đổi giữa các khung hình nhưng việc mà để cân bằng giữa hiệu suất và độ chính xác phù hợp khi chạy thử các video thì ở mô hình có 60 chuỗi khung hình sẽ ổn định nhất



Hình 5.1 Biểu đồ Training và Validation Loss



Hình 5.2 Biểu đồ Training và Validation Accuracy



Hình 5.3 Confusion Matrix: Phân loại Video Fake và Real

Kết quả độ chính xác ở 20 epochs thì mô hình đạt được :

- **Accuracy** $\approx 92.3\%$
- **Precision** $\approx 92.3\%$
- **Recall** $\approx 84.2\%$
- **F1-Score** $\approx 88.1\%$

5.2. Kết quả khi so sánh với mô hình ResNeXt50

Để đánh giá mô hình trực quan hơn thì em đã so sánh với mô hình ResNeXt50 [19]

Model Name	Frames	Accuracy ResNeXt50	Accuracy ResNeX101
model_81_acc_10_frames_final_data.pt	10	81.23	84.21
model_87_acc_20_frames_final_data.pt	20	87.79	88.74
model_89_acc_40_frames_final_data.pt	40	89.34	90.37
model_89_acc_60_frames_final_data.pt	60	90.5	92.3
model_91_acc_80_frames_final_data.pt	80	91.4	92.83

Bảng 5.1 So sánh kết quả độ chính xác của 2 mô hình

CHƯƠNG 6. KẾT LUẬN VÀ PHƯƠNG HƯỚNG PHÁT TRIỂN TRONG TƯƠNG LAI

6.1. Tổng kết các kết quả đạt được

Khi sử dụng một mô hình mạnh hơn kết hợp thêm các kỹ thuật như LSTM 2 Chiều ,tầng chú ý(Attention layer) để tăng sự độ chính xác cho mô hình thì nhận được các kết quả khá tốt khi so sánh với mô hình ResNeXt50 kết hợp với LSTM là khoảng hơn 1% .

6.2. Các hạn chế và thách thức còn tồn tại

Hiện tại mô hình đang còn nhiều thiếu sót cần phải qua quá trình kiểm thử và sử dụng trong một thời gian , mô hình còn nhiều hạn chế khi so sánh với các mô hình mạng CNN mới và mạnh mẽ hiện nay. Việc tối ưu được hiệu suất đồng thời đảm bảo được độ chính xác là một bài toán khó khi mà hai yếu tố này có tác động trái chiều. Việc tăng chuỗi khung hình giúp tăng tỷ lệ chính xác nhưng khả năng phân tích các video đầu vào của người dùng sẽ rất lâu.

Việc sử dụng mô hình mới ResNeXt101 cần đòi hỏi lớn hơn về bộ nhớ nhiều và cần một số yêu cầu để có thể tạo ra môi trường python giúp mô hình có thể hoạt động được như CUDA,pytorch,cmake, dlib và một số thư viện python khác cần được tải về để có thể sử dụng cho việc nhận diện . Chính vì vậy cần đòi hỏi người dùng phải đáp ứng được các phần cứng và mềm để có thể hoạt động mô hình trong trường hợp chạy cục bộ.

6.3. Phương hướng phát triển trong tương lai

Để có thể phát triển được mô hình trong tương lai và có thể bắt kịp được việc các video deepfake ngày càng trở nên mạnh mẽ thì em xin đề xuất những phương hướng phát triển để mô hình có thể áp dụng trong thực tế

6.3.1. Cân bằng hiệu suất và độ chính xác ổn định:

Tối ưu hóa mô hình:

- Áp dụng kỹ thuật pruning để giảm số lượng tham số mà vẫn duy trì độ chính xác.

- Sử dụng quantization để giảm kích thước mô hình và tăng tốc độ suy luận.

Triển khai hiệu quả:

- Tận dụng các framework tối ưu hóa như TensorRT hoặc ONNX Runtime.
- Xem xét việc sử dụng edge computing để giảm độ trễ trong các ứng dụng thời gian thực.

Kiểm tra độ ổn định:

- Thực hiện kiểm tra cross-validation kỹ lưỡng để đảm bảo mô hình hoạt động ổn định trên các tập dữ liệu khác nhau.
- Áp dụng kỹ thuật ensemble để tăng tính ổn định và độ tin cậy của kết quả.

6.3.2. Kết hợp thêm các mô hình CNN mới hơn:

EfficientNet:

- Tận dụng khả năng mở rộng hiệu quả của EfficientNet để cân bằng giữa độ sâu, chiều rộng và độ phân giải.
- Thử nghiệm với các phiên bản khác nhau từ B0 đến B7 để tìm ra cấu hình tối ưu.

Vision Transformer (ViT):

- Khai thác khả năng học các mối quan hệ dài hạn trong hình ảnh của ViT.
- Xem xét các biến thể như DeiT hoặc Swin Transformer để cải thiện hiệu suất trên bài toán phát hiện deepfake.

Kiến trúc lai:

- Thử nghiệm với các mô hình kết hợp ưu điểm của CNN và Transformer như ConvNeXt hoặc CoAtNet.

6.3.3. Thu thập và bổ sung tập dữ liệu đa dạng hơn:

Đa dạng hóa nguồn dữ liệu:

- Thu thập video deepfake từ nhiều nguồn khác nhau, bao gồm cả các phương pháp tạo deepfake mới nhất.
- Đảm bảo dữ liệu bao gồm nhiều độ phân giải, điều kiện ánh sáng và góc quay khác nhau.

Tăng cường dữ liệu:

- Áp dụng các kỹ thuật augmentation tiên tiến như AutoAugment hoặc RandAugment.
- Sử dụng GAN để tạo ra các mẫu deepfake mới, giúp mô hình học được các đặc trưng tinh vi hơn.

Dữ liệu đặc biệt:

- Tập trung vào các trường hợp khó, như deepfake chất lượng cao hoặc các video có sự can thiệp tinh vi.
- Xây dựng một tập dữ liệu "adversarial" để kiểm tra và cải thiện độ mạnh mẽ của mô hình.

TÀI LIỆU THAM KHẢO

- [1] S. ADEE, “What Are Deepfakes and How Are They Created?,” *IEEE Spectrum*, 2020.
- [2] N. a. B. V. a. G. R. Khatri, “A Comparative Study: Deepfake Detection Using Deep-learning,” trong *2023 13th International Conference on Cloud Computing, Data Science & Engineering (Confluence)*, 2023, pp. 1-5.
- [3] Mohsen, Saeed and Ali, Anas M. and El-Rabaie, El-Sayed M. and ElKaseer, Ahmed and Scholz, Steffen G. and Hassan, Ashraf Mohamed Ali, “Brain Tumor Classification Using Hybrid Single Image Super-Resolution Technique With ResNext101_32× 8d and VGG19 Pre-Trained Models,” *IEEE Access*, p. 11, 2023.
- [4] A. a. B. B. R. a. A. H. J. Arini, “Quick Classification of Xception And Resnet-50 Models on Deepfake Video Using Local Binary Pattern,” trong *2021 International Seminar on Machine Learning, Optimization, and Data Science (ISMODE)*, 2022, pp. 254-259.
- [5] Y. a. Z. S. Al-Dhabi, “2021 IEEE International Conference on Computer Science, Artificial Intelligence and Electronic Engineering (CSAIEE),” trong *Deepfake Video Detection by Combining Convolutional Neural Network (CNN) and Recurrent Neural Network (RNN)*, 2021, pp. 236-241.
- [6] S. J. Lawrence, “scalertopics,” What is LSTM? - Introduction to Long Short-Term Memory, 2023. Available: <https://www.scaler.com/topics/deep-learning/lstm/>.
- [7] A. a. C. D. a. V. L. a. R. C. a. T. J. a. N. M. Rössler, “FaceForensics++: Learning to Detect Manipulated Facial Images,” trong *2019 IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019, pp. 1-11.

- [8] “Deepfake Detection Challenge dataset,” AWS, Facebook, Microsoft, 2019.
Available: <https://www.kaggle.com/c/deepfake-detection-challenge/overview>.
- [9] Y. a. Y. X. a. S. P. a. Q. H. a. L. S. Li, “Celeb-DF: A Large-Scale Challenging Dataset for DeepFake Forensics,” trong *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 3204-3213.
- [10] J. a. B. V. Langr, *GANs in Action: Deep learning with Generative Adversarial Networks*, 2019.
- [11] N. Hưng, “Tổng quan về Deep Learning từ A-Z,” . Available:
<https://vietnix.vn/deep-learning-la-gi/>.
- [12] H. Nguyễn, “Các kỹ thuật Deep Learning,” . Available: <https://vietnix.vn/deep-learning-la-gi/#cac-ky-thuat-deep-learning>.
- [13] Z. C. L. L. J. S. Aston Zhang, *Dive into Deep Learning*, 2023.
- [14] L. a. Z. Y. Gonog, “A Review: Generative Adversarial Networks,” trong *{2019 14th IEEE Conference on Industrial Electronics and Applications (ICIEA)*, 2019 pp. 505-510.
- [15] K. a. Z. X. a. R. S. a. S. J. He, “{Deep Residual Learning for Image Recognition,” trong *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770-778.
- [16] S. a. G. R. a. D. P. a. T. Z. a. H. K. Xie, “Aggregated Residual Transformations for Deep Neural Networks,” trong *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [17] NVIDIA, “pytorch,” NVIDIA, 2019. Available:
https://pytorch.org/hub/nvidia_deeplearningexamples_resnext/index.html.
- [18] H. a. P. R. a. S. A. A. Lu, “2023 IEEE International Conference on Multimedia and Expo (ICME),” trong *LA-layer: General local attention layer for full attention networks*, 2023, pp. 2057-2062.

- [19] H. P. P. Jay Patel, “Deepfake detection using Deep Learning (ResNext and LSTM),” Github, 2019. Available:
https://github.com/abhijitjadhav1998/Deepfake_detection_using_deep_learning